

=====

TABLE OF CONTENTS

=====

- 1. Platform Overview
- 2. Architecture
- 3. Getting Started
- 4. Authentication & Security
- 5. API Reference
- 6. Deployment Guide
- 7. Troubleshooting
- 8. Performance Optimization

=====

1. PLATFORM OVERVIEW

=====

1.1 What is AcmeCloud?

AcmeCloud is ACME Corporation's enterprise cloud platform designed for deploying, managing, and scaling business applications. It provides infrastructure-as-a-service (IaaS) and platform-as-a-service (PaaS) capabilities.

- 1.2 Key Features
- Auto-scaling compute instances
  - Managed database services (PostgreSQL, MongoDB, Redis)
  - Container orchestration with Kubernetes
  - Built-in CI/CD pipelines
  - Real-time monitoring and alerting
  - Multi-region deployment
  - Automated backup and disaster recovery

1.3 Supported Technologies

Languages: Python, Java, Node.js, Go, Ruby, .NET

Databases: PostgreSQL 15+, MongoDB 7+, Redis 7+, MySQL 8+

Containers: Docker, Kubernetes 1.28+

Message Queues: RabbitMQ, Apache Kafka

=====

2. ARCHITECTURE

=====

2.1 System Components

The AcmeCloud platform consists of four primary layers:

Control Plane: Manages cluster lifecycle, scheduling, and API gateway

Compute Layer: Runs application workloads in isolated containers  
Data Layer: Provides managed database and storage services  
Network Layer: Handles load balancing, DNS, and inter-service communication

## 2.2 High Availability

The platform runs across three availability zones. Each zone is a physically separated data center. Database replicas are distributed across all zones with automatic failover. Recovery Point Objective (RPO) is 1 minute, Recovery Time Objective (RTO) is 5 minutes.

## 2.3 Security Architecture

All inter-service communication uses mutual TLS (mTLS). Each tenant is isolated in its own network namespace. Secrets are managed through HashiCorp Vault integration. All API calls are authenticated and rate-limited.

# =====

## 3. GETTING STARTED

# =====

### 3.1 Account Setup

1. Visit [cloud.acme-corp.com](https://cloud.acme-corp.com) and click "Create Account"
2. Verify your email address
3. Complete organization setup
4. Add a payment method (credit card or invoice)
5. Install the AcmeCloud CLI

### 3.2 CLI Installation

macOS: `brew install acmecloud-cli`  
Linux: `curl -sSL https://get.acmecloud.io | bash`  
Windows: `winget install AcmeCorp.AcmeCloud`

### 3.3 First Deployment

To deploy your first application:

```
acme login
acme init my-app
acme deploy
```

This creates a new application with default settings (1 instance, 512MB RAM, auto-scaling disabled).

### 3.4 Resource Tiers

Starter: 1 vCPU, 512MB RAM, 10GB storage · \$0 (free tier)  
Basic: 2 vCPU, 2GB RAM, 50GB storage · \$25/month  
Professional: 4 vCPU, 8GB RAM, 200GB storage · \$99/month  
Enterprise: 16 vCPU, 32GB RAM, 1TB storage · \$399/month

# =====

## 4. AUTHENTICATION & SECURITY

# =====

### 4.1 API Keys

API keys are used for programmatic access. Keys can be scoped to specific permissions (read, write, admin). Keys should be rotated every 90 days. Never embed API keys in source code.

## 4.2 OAuth 2.0 Integration

AcmeCloud supports OAuth 2.0 for single sign-on (SSO). Supported identity providers: Google Workspace, Microsoft Azure AD, Okta, Auth0. SAML 2.0 is also supported for enterprise customers.

## 4.3 Role-Based Access Control (RBAC)

Predefined roles:

Owner: Full access, billing management

Admin: Resource management, user management

Developer: Deploy, manage applications

Viewer: Read-only access to dashboards and logs

## 4.4 Two-Factor Authentication

2FA is mandatory for all admin and owner accounts. Supported methods: authenticator app (TOTP), hardware security keys (FIDO2), SMS (deprecated, not recommended).

# 5. API REFERENCE

## 5.1 Base URL

Production: <https://api.acmecloud.io/v4>

Sandbox: <https://sandbox-api.acmecloud.io/v4>

## 5.2 Rate Limits

Free tier: 100 requests per minute

Basic: 500 requests per minute

Professional: 2,000 requests per minute

Enterprise: 10,000 requests per minute

## 5.3 Common Endpoints

POST /apps · Create a new application

GET /apps/{id} · Get application details

PUT /apps/{id}/scale · Scale application instances

POST /apps/{id}/deploy · Deploy new version

GET /apps/{id}/logs · Stream application logs

DELETE /apps/{id} · Delete application

## 5.4 Error Codes

400: Bad Request · Invalid parameters

401: Unauthorized · Invalid or expired API key

403: Forbidden · Insufficient permissions

404: Not Found · Resource does not exist

429: Too Many Requests · Rate limit exceeded

500: Internal Server Error · Contact support

## =====

## 6. DEPLOYMENT GUIDE

## =====

### 6.1 Blue-Green Deployment

AcmeCloud supports zero-downtime deployments using blue-green strategy. The new version is deployed alongside the existing version. Traffic is gradually shifted from old to new (canary). If health checks fail, traffic is automatically rolled back.

### 6.2 Scaling

Auto-scaling is configured through scaling policies:

Min instances: 1 (default)

Max instances: 50

Scale-up threshold: CPU > 70% for 5 minutes

Scale-down threshold: CPU < 30% for 10 minutes

Cooldown period: 3 minutes between scaling events

### 6.3 Environment Variables

Environment variables are managed through the dashboard or CLI:

acme config set DATABASE\_URL=postgres://...

acme config set --secret API\_SECRET=sk-xxx

Variables marked as --secret are encrypted and never displayed in logs.

## =====

## 7. TROUBLESHOOTING

## =====

### 7.1 Application Won't Start

Check the application logs: `acme logs --tail 100`

Common causes:

- Missing environment variables
- Port conflict (ensure app binds to PORT env variable)
- Insufficient memory (check resource tier)
- Failed health checks (verify /health endpoint returns 200)

### 7.2 High Latency

Check the monitoring dashboard for CPU and memory usage.

Common solutions:

- Scale up to a higher resource tier
- Enable auto-scaling
- Optimize database queries
- Enable CDN for static assets
- Check network egress patterns

### 7.3 Database Connection Issues

Verify the DATABASE\_URL environment variable is correct.

Check that the database is in the same region as the application.

Ensure connection pooling is enabled (max 20 connections recommended).

Check if the database has reached its storage limit.

=====

## 8. PERFORMANCE OPTIMIZATION

=====

### 8.1 Caching

Enable Redis caching for frequently accessed data. Default TTL should be 300 seconds (5 minutes). Use cache-aside pattern for database reads. Invalidate cache on write operations.

### 8.2 CDN Configuration

Static assets should be served through AcmeCloud CDN. CDN is available in 120+ edge locations globally. Enable compression for text-based assets (HTML, CSS, JS). Set appropriate Cache-Control headers.

### 8.3 Database Optimization

Use connection pooling (PgBouncer for PostgreSQL). Add indexes for frequently queried columns. Use read replicas for analytics queries. Enable query logging to identify slow queries (> 100ms).

### 8.4 Monitoring

AcmeCloud Monitoring provides:

- Real-time CPU, memory, and network metrics
- Custom alert rules with email, Slack, PagerDuty notifications
- Distributed tracing for microservices
- Error tracking with stack traces
- Monthly performance reports

=====

END OF DOCUMENT

=====